

Protidiner Sokal — Full System Architecture Documentation

Prepared By: Shah Mozzem Remal (Full Stack Developer)

1. Project Overview

Project Name

Protidiner Sokal

Project Type

Modern Digital News Portal & Media Publishing Platform

Objective

Build a scalable, SEO-optimized, high-performance Bangladeshi digital news platform capable of handling:

- High traffic
 - Real-time breaking news
 - Multi-role newsroom workflows
 - Advertisement management
 - Audience engagement
 - Future AI integration
 - Mobile-first experience
-

2. Technology Stack

Frontend

Framework

- Next.js (TypeScript)

UI System

- ShadCN UI
- Tailwind CSS

Frontend Responsibilities

- Public website rendering
 - SEO rendering
 - Category pages
 - News details pages
 - Authentication UI
 - Admin dashboard UI
 - Reporter dashboard
 - Advertisement management UI
 - Comment system
 - Bookmark system
 - User profile system
-

Backend

Runtime

- Node.js (TypeScript)

Framework

- Express.js

Responsibilities

- REST API services
 - Authentication & authorization
 - News workflow management
 - Comment moderation
 - Advertisement APIs
 - Analytics APIs
 - Notification services
 - File upload processing
 - Redis caching
 - Background processing
-

Database

Database Engine

- MySQL

ORM

- Prisma ORM

Responsibilities

- Relational data management
 - Schema migration
 - Query optimization
 - Type-safe database operations
-

Media Storage

Cloudinary

Responsibilities

- Image storage
 - Video storage
 - CDN delivery
 - Image optimization
 - Responsive media transformation
 - WebP/AVIF conversion
-

Cache Layer

Redis

Responsibilities

- API caching
 - Homepage caching
 - Session storage
 - Trending news cache
 - Rate limiting
 - Real-time event buffering
-

3. High-Level System Architecture

Users



Next.js Frontend



Express.js API Server



Prisma ORM



MySQL Database

Additional Services:

- Redis Cache
 - Cloudinary Media Storage
 - Notification Service
 - Analytics Service
-

4. Core Modules

Public News Portal

Features:

- Homepage
 - Category pages
 - Subcategory pages
 - News details pages
 - Trending news
 - Breaking news ticker
 - Video news
 - Photo gallery
 - Search system
 - Live updates
 - Bookmark system
 - Reactions & comments
-

CMS / Admin Panel

Features:

- Dashboard analytics
- News management
- Category management
- User management
- Role management
- Advertisement management
- Comment moderation
- SEO management
- Media management
- Breaking news control

Authentication System

Features:

- Login
 - Registration
 - Email verification
 - Password reset
 - Role-based access control
 - Session management
 - JWT authentication
-

5. RBAC (Role-Based Access Control)

1. Admin

Permissions

- Full system access
 - User management
 - Role management
 - Site settings
 - News management
 - Delete any resource
 - Advertisement control
 - Analytics access
 - SEO management
 - Moderation control
-

2. Editor

Permissions

- Approve/reject news
- Read all news
- Update all news
- Publish news
- Unpublish news
- Delete news
- Manage featured news
- Manage breaking news

3. Journalist / Reporter

Permissions

- Create draft news
 - Read own news
 - Update own drafts
 - Submit news for review
 - Upload media
 - View publishing status
-

4. Advertisement Manager

Permissions

- Create ads
 - SEO Manage
 - Manage banners
 - Schedule advertisements
 - Enable/disable ads
 - Track ad performance
 - Homepage ad placement management
-

5. Moderator

Permissions

- Read comments
 - Approve comments
 - Delete comments
 - Block users
 - Moderate reactions
 - Handle reported content
-

6. Reader/User

Permissions

- Read news
- Share news
- Comment on news
- React to news

- Bookmark news
 - Save articles
 - Follow categories
-

6. News Workflow

Reporter Creates Draft



Submit for Review



Editor Reviews



Approved / Rejected



Published



Visible to Public

7. Database Architecture

Core Tables

Users

- ❖ Stores all platform users.

Roles

- ❖ Stores RBAC role definitions.

Permissions

- ❖ Stores system permissions.

UserRoles

- ❖ Maps users to roles.

News

- ❖ Stores news articles.

Categories

- ❖ Stores main categories.

SubCategories

- ❖ Stores subcategories.

Tags

- ❖ Stores tags.

NewsTags

- ❖ Many-to-many relation.

Comments

- ❖ Stores article comments.

Reactions

- ❖ Stores reactions.

Bookmarks

- ❖ Stores saved articles.

Advertisements

- ❖ Stores ad information.

Media

- ❖ Stores uploaded media.

BreakingNews

- ❖ Stores breaking headlines.

SEO_Metadata

- ❖ Stores SEO metadata.

Notifications

- ❖ Stores user notifications.

Analytics

- ❖ Stores analytics records.

8. Suggested Prisma Models

User

```
model User {
  id          String   @id @default(cuid())
  name        String
  email       String   @unique
  password    String
  roleId      String
  role        Role     @relation(fields: [roleId], references: [id])
  createdAt   DateTime @default(now())
}
```

Role

```
model Role {
  id      String @id @default(cuid())
  name    String @unique
  users   User[]
}
```

News

```
model News {
  id          String   @id @default(cuid())
  title       String
  slug        String   @unique
  content     String
  featuredImage String?
  status      NewsStatus
  authorId    String
  categoryId  String
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}
```

9. News Status Lifecycle

DRAFT
PENDING_REVIEW
APPROVED
PUBLISHED
REJECTED

UNPUBLISHED
ARCHIVED

10. API Architecture

API Versioning

/api/v1/

11. Authentication APIs

Auth Routes

POST /api/v1/auth/register
POST /api/v1/auth/login
POST /api/v1/auth/logout
POST /api/v1/auth/refresh
POST /api/v1/auth/forgot-password
POST /api/v1/auth/reset-password

12. News APIs

GET /api/v1/news
GET /api/v1/news/:slug
POST /api/v1/news
PATCH /api/v1/news/:id
DELETE /api/v1/news/:id
POST /api/v1/news/:id/submit
POST /api/v1/news/:id/approve
POST /api/v1/news/:id/reject
POST /api/v1/news/:id/publish

13. Category APIs

GET /api/v1/categories
POST /api/v1/categories
PATCH /api/v1/categories/:id
DELETE /api/v1/categories/:id

14. Comment APIs

```
POST    /api/v1/comments
GET      /api/v1/comments/:newsId
DELETE   /api/v1/comments/:id
PATCH   /api/v1/comments/:id/moderate
```

15. Advertisement APIs

```
POST    /api/v1/ads
GET      /api/v1/ads
PATCH   /api/v1/ads/:id
DELETE   /api/v1/ads/:id
```

16. Frontend Architecture

Recommended App Structure

```
src/
├── app/
├── components/
├── features/
├── services/
├── hooks/
├── store/
├── types/
├── utils/
├── middleware/
├── lib/
├── styles/
└── config/
```

17. Frontend Features

Public Features

- Homepage
- Category pages
- Search page
- Author page
- Trending page

- Breaking news section
 - Related news
 - Bookmark system
 - Social sharing
 - Infinite scrolling
 - Lazy loading
-

18. Backend Architecture

```
├── prisma/
|   ├── schema.prisma
|   ├── seed.ts
|   └── migrations
├── src/
|   ├── config
|   ├── middleware
|   ├── module
|   ├── router
|   ├── types
|   ├── utils
|   ├── app.ts
|   └── index.ts
├── .env
├── .gitignore
├── package.json
└── tsconfig.json
```

Dashboard Features

Admin Dashboard

- Analytics overview
- User management
- News statistics
- Ad revenue analytics
- Traffic monitoring

Editor Dashboard

- Pending review queue
- Published news
- Rejected news

Reporter Dashboard

- Drafts
 - Pending review articles
 - Published news
-

19. SEO Architecture

SEO Features

Dynamic Metadata

- Dynamic titles
- Meta descriptions
- Open Graph tags
- Twitter/Facebook cards

Structured Data

- JSON-LD schema
- Article schema
- Breadcrumb schema

Technical SEO

- Sitemap.xml
 - Robots.txt
 - Canonical URLs
 - RSS feeds
-

20. URL Structure

SEO-Friendly URL Example

`/category/news-title-slug`

Example:

`/technology/apple-launches-new-device`

21. Performance Optimization

Frontend Optimization

- Server-side rendering
 - Incremental static regeneration
 - Dynamic imports
 - Code splitting
 - Image optimization
 - Lazy loading
-

Backend Optimization

- Redis caching
 - Query optimization
 - Pagination
 - Compression middleware
 - API rate limiting
-

Database Optimization

- Proper indexing
 - Optimized relations
 - Query batching
 - Database connection pooling
-

22. Redis Caching Strategy

Cache Targets

- Homepage news

- Trending news
 - Breaking news
 - Popular categories
 - Advertisement blocks
 - Search suggestions
-

Cache Expiration

Homepage Cache: 5 minutes
Trending News: 10 minutes
Category Pages: 15 minutes
Ads Cache: 30 minutes

23. Media Management

Cloudinary Usage

Store

- News images
 - Reporter profile photos
 - Advertisement banners
 - Video thumbnails
 - Gallery images
-

Image Optimization

- Auto compression
 - Responsive sizes
 - WebP conversion
 - Lazy loading
-

24. Security Architecture

Security Features

- JWT authentication
- Password hashing
- Rate limiting
- XSS protection

- CSRF protection
 - Helmet.js security headers
 - Input validation
 - SQL injection protection
-

25. Deployment Architecture

Recommended Production Deployment

Nginx
↓
Next.js Frontend
↓
Express.js Backend
↓
Redis
↓
MySQL

26. Recommended Hosting

Frontend Hosting

- Vercel
- VPS

Backend Hosting

- VPS
- DigitalOcean
- AWS EC2

Database Hosting

- Managed MySQL
 - VPS MySQL
-

27. CI/CD Pipeline

Recommended Workflow

GitHub
↓

GitHub Actions



Testing



Build



Deployment

28. Testing Strategy

Frontend Testing

- React Testing Library
- Playwright

Backend Testing

- Jest
- Supertest

Types of Tests

- Unit tests
 - Integration tests
 - API tests
 - End-to-end tests
-

29. Future Features

Recommended Future Integrations

- AI-generated summaries
- AI headlines
- Personalized news feed
- Voice news reader
- Mobile apps
- Push notifications
- Real-time live blogging
- Subscription plans

- Premium content
 - Multi-language support
-

30. Mobile Application Strategy

Recommended Stack

- React Native OR
- Flutter

Features:

- Push notifications
 - Offline reading
 - Saved news
 - Personalized feed
-

31. Search System

Recommended Search Engine

Initial Phase

- MySQL Full-text Search

Advanced Phase

- Elasticsearch OR
 - Meilisearch
-

32. Analytics System

Metrics to Track

- Total page views
- Unique visitors
- Trending categories
- User engagement
- Ad clicks
- Read time

- Bounce rate
-

33. Recommended Development Standards

Coding Standards

- TypeScript strict mode
 - ESLint
 - Prettier
 - Husky pre-commit hooks
-

Git Workflow

```
main
├── development
├── staging
└── feature/*
```

34. Environment Variables

Backend

```
DATABASE_URL=
JWT_SECRET=
REDIS_URL=
CLOUDINARY_API_KEY=
CLOUDINARY_SECRET=
```

Frontend

```
NEXT_PUBLIC_API_URL=
NEXT_PUBLIC_SITE_URL=
```

35. Final Notes

Protidiner Sokal should not be treated as a traditional news website.

It should be designed as:

- A scalable digital media platform
- A high-performance publishing engine
- A modern SEO-first content system
- A social media optimized distribution platform
- A future-ready AI-enabled newsroom ecosystem